

Éxito

Para acceder a los datos a través de las relaciones, la relación tiene que tener cardinalidad (1,1) porque sino nose cual meteraigo. y si es (0,1) no puedo tampoco y menos traer el identificador

démion

bonzi

Modelado conceptual

- (1,1): Al menos uno, pero solo uno
- (1,S): Al menos uno, puede ser ambos
- (P,1): Alguno, pero solo uno
- (P,S): Alguno, puede ser ambos

Modelado logico

- Resolver jerarquias $\rightarrow$ Porque no se aceptan hierarquias en NF
- Resolver Atributos compuestos $\rightarrow$ Porque no se aceptan registros
- Resolver Atributos holivalentes $\rightarrow$ Porque no se aceptan listas

Modelado fisico

- $B = (0,1) \xrightarrow{(1,1)} A$
- $B = (1,1) \xrightarrow{(1,1)} A$
- $B = (0,N) \xrightarrow{(1,1)} A$
- $B = (1,N) \xrightarrow{(1,1)} A$
- $B = (x,x) \xrightarrow{(1,1)} A$
- $B = (\underline{idB}, \alpha B)$
- $A = (\underline{idA}, idB(fN), \alpha A)$

- $A = (0,1) \xrightarrow{(0,1)} B \xrightarrow{(1,N)} A \rightarrow B = (\underline{idA(fN)}, idB(fN))$
(el minimo porque aparece 1 sola vez o no aparece)

- $A = (1,N) \xrightarrow{(1,N)} B \xrightarrow{(0,N)} A \rightarrow B = (\underline{idA(fN)}, idB(fN))$

- $RECURSIVA = B = (\underline{idA(fN)}, idA REC(fN))$

Si uso **LEFT JOIN** no usar **WHERE** porque me rompe los **NULLS**, usar **ON**

Actualización (**UPDATE** tabla **SET** atributo = valor **WHERE** atributo = valor)

Eliminación (**DELETE FROM** tabla **WHERE** condicion) (tengo que eliminar de abajo hacia arriba)

UNION, **INTERSECT** y **EXCEPT**, eliminan duplicados

El **%** del **LIKE** se usa para completar el string.

de michi

Ej: que termine en 'EZ' **LIKE** '%EZ', que empiece con 221 **LIKE** '221%' bonzi

LEFT JOIN lo utilizo cuando un dato que necesito puede ser **NULL** y lo quiero tener en cuenta.

Ej: Producto **LEFT JOIN** Detalle (el producto puede no haberse vendido nunca)

GROUP BY usa cuando aparece: por cada, por cada, cuantos, total de.

JOIN → **GROUP BY** → función (**COUNT**, **SUM**, **AVG**)

el **GROUP BY** resuelve varias filas en una sola

Si uso una función y en el **SELECT** tengo varios datos necesito **GROUP BY**

Agregación (**INSERT INTO** tabla() **VALUES** ()

Relación entre consultas (**EXIST** / **NOT EXIST**) → necesitan relación

Ej: (d. ticket = f. ticket) → facturas sin producto 'x'

HAVING uso cuando aparece que 'sobre x' (después de agrupar)

El orden de los **INNER JOIN** no importa, el de los **LEFT RIGHT** si

SELECT DISTINCT: ¿estoy uniendo con una tabla "detalle" (1 a muchos)?

Podador → Poda (1 podador puede tener muchas podas)

y estoy mostrando datos del "lado 1" entonces uso **DISTINCT**

COUNT (*) → cuenta filas • **COUNT (columna)** → cuenta valores no null

WHERE me elimina filas con null, esto es importante cuando uso **LEFT JOIN**

Doble **NOT EXIST**, ej listar clientes que viajaron con todos los géneros

o sea: traer clientes para los cuales **NO** exista una agencia con la que **NO** viajaron